

DL for ENG

LECTURE 2.1 · RECORDED

# Scientific Python

---

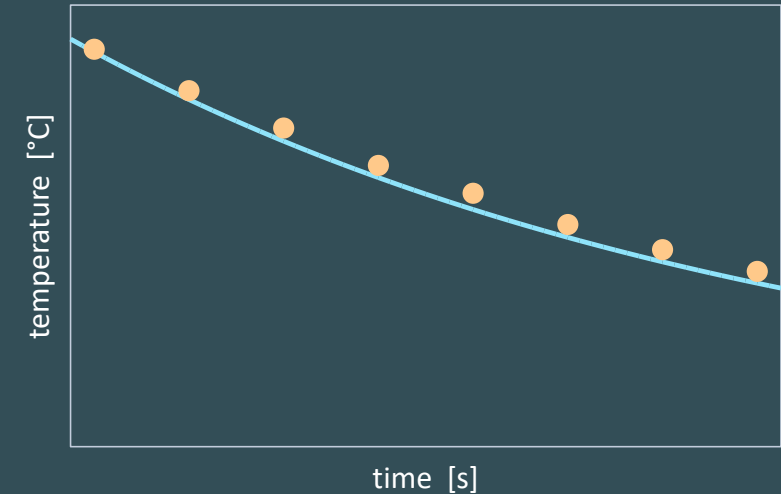
**Remus Teodorescu**

*With support from Research Assistant Noman Khan -  
nomank@energy.aau.dk*

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

PANDAS · SOLVE\_IVP · CURVE\_FIT



*An aluminium block cooling in a room, logged every two seconds. It arrives as a messy file, is solved as a differential equation, and is fitted as a two-parameter model. That fitted model is the baseline every network here has to beat.*

# Learning Objectives

## SECTIONS, IN ORDER

- 1 **Pandas** — the DataFrame, reading a real file, selecting, missing data, aggregating
- 2 **Solving** — the state form, solve\_ivp, and how to know the answer is right
- 3 **Fitting** — a model with two knobs, curve\_fit, the covariance, and the baseline

*Everything here sits on the arrays of the last lecture. Pandas adds labels and missing values, SciPy adds solvers, and underneath both it is still NumPy.*

## READ, AND WHAT EACH ANSWERS

|                        |                                              |
|------------------------|----------------------------------------------|
| <b>pandas guide</b>    | how do I get a real file into an array?      |
| <b>SciPy integrate</b> | how do I solve it, and is the answer right?  |
| <b>SciPy optimize</b>  | what are the parameters, and how well known? |

# Recap

## 1 – 5 · THE ARRAY

- 1 What is a NumPy array, how does it differ from a list, and where does its speed come from? *NumPy basics, arrays*
- 2 What is a dtype, when is it fixed, and what happens when you assign a float into an integer array? *NumPy basics, types; Ex\_01 notebook 02*
- 3 What does a shape tuple mean, how do you read it, and what is the habit this lecture asks for? *NumPy basics, shape*
- 4 What does indexing with an integer do to an axis that slicing with a colon does not? *NumPy basics, indexing*
- 5 What is vectorisation, roughly how much faster is it, and why is a comprehension not vectorised? *NumPy basics, performance; Ex\_01 notebook 02*

## 6 – 10 · BROADCASTING

- 6 What is the broadcasting rule, in one sentence, and which half of it do people forget? *NumPy basics, broadcasting*
- 7 How can subtracting two arrays of a hundred numbers give ten thousand, and why does nothing raise? *NumPy broadcasting; Ex\_01 notebook 03*
- 8 What do None, reshape, ravel and squeeze each do, and which says you meant to broadcast? *NumPy basics, indexing*
- 9 In a reduction, what does the axis argument name, and what shape comes back? *NumPy basics, reductions*
- 10 What makes a figure defensible, and what does a residual plot show that an error number does not? *Matplotlib user guide; Ex\_01 notebook 03*

# DataFrame

**DataFrame** — a two-dimensional table with a name on every column and an index on every row. Each column is a NumPy array with its own dtype, so one table can hold a timestamp, a float channel and a status string. Think of it as a dictionary of columns rather than a list of rows, because that is how it is stored.

## AND ONE COLUMN AT A TIME, NOT ONE ROW AT A TIME

The index is the part with no NumPy equivalent. It labels the rows, it need not be integers, and when it is a time it unlocks resampling and interpolation for nothing.

Everything from the last lecture still applies underneath. A column is an array, broadcasting works, and the shape rule is unchanged. Pandas adds labels and missing values on top.

## THE OBJECT

| index | time_s | temp_C | status |
|-------|--------|--------|--------|
| 0     | 0.0    | 80.05  | OK     |
| 1     | 2.0    | 78.11  | OK     |
| 2     | 4.0    | NaN    | FAULT  |
| 3     | 6.0    | 74.90  | OK     |

## THREE THINGS TO CALL FIRST

```
df.head()      the first five rows
df.dtypes      one dtype per column
df.describe()  count, mean, min, max
# and df.shape, every time
```

# Reading a CSV

**read\_csv** — the one function that turns a delimited text file into a DataFrame. In the easy case that is the whole story. In the real case, where the file came off a logger, six of its arguments do almost all the work.

## ONE FUNCTION DOES ALMOST ALL OF IT

`sep`, when the file is not comma-separated, because European exports are usually semicolons. `skiprows`, for the header block a logger writes. `decimal`, for the comma decimal separator.

`na_values`, for whatever the instrument writes when a channel drops out. `parse_dates`, for timestamp columns. And `usecols`, when the file has forty channels and you want three.

## THE ARGUMENTS THAT MATTER

```
import pandas as pd

df = pd.read_csv(
    "cooling_log.csv",
    sep=";",          # not a comma
    decimal=".",      # European decimals
    skiprows=2,       # logger preamble
    na_values=["", "SENSOR FAULT"],
    usecols=["time", "temp_C"],
)

df.dtypes           # look, every time
```

*An object dtype where you expected a float is a file problem, not a pandas problem.*

# Selecting

**loc and iloc** — the two indexers, and the difference matters. loc selects by label, and its slices include their endpoint, because a label has no arithmetic. iloc selects by position, and its slices exclude the endpoint exactly as an array's do.

## TWO INDEXERS, AND WHICH IS WHICH

Mixing them up gives an off-by-one that does not raise. The rule worth adopting: iloc when you are thinking about positions, loc when you are thinking about names or times.

Boolean masks work as they did last lecture and are the right way to express a condition: the temperatures above fifty degrees, the rows where the status column says running.

## THE FOUR FORMS

```
df["temp_C"]           a Series
df[["time_s", "temp_C"]] a DataFrame

df.iloc[0:5]           by position, [0,5)
df.loc[0:5]            by label, [0,5]

df.loc[df["temp_C"] > 50.0]
df.loc[df["status"] == "OK", "temp_C"]

t = df["time_s"].to_numpy()
T = df["temp_C"].to_numpy()
```

*The last two lines are the exit door. Everything after slide ten is NumPy.*

# Missing Data

**Missing data** — a value the instrument did not give you, held as not-a-number. It propagates, so any arithmetic involving it is also not-a-number. That is a feature: a missing value cannot quietly become a wrong one.

## AND ONE DISHONEST ONE

There are three defensible responses, and which you choose depends on why the value is missing rather than on convenience. Drop the row, interpolate across a short gap, or carry the last reading forward.

The dishonest option is filling with zero. A missing temperature is not zero degrees, and a zero drags every mean, every fit and every plot towards itself while looking like data.

**TALK • 2 MIN** How would you decide what to do with a missing measurement, in your own words? Then, in the three options on the right, which one would you choose for a thermocouple that dropped out for two seconds, and what would you write in the report?

## THE THREE

**dropna()**

the reading is absent — drop the row

**interpolate()**

smooth quantity, short gap

**keep the NaN**

use mean(), which skips it

```
df["temp_C"].isna().sum()      1
clean = df.dropna(subset=["temp_C"])
len(df), len(clean)           6, 5
```

## THE ONE TO REFUSE

fillna(0.0). A missing temperature is not zero degrees.

# Aggregating

**groupby** — an operation that splits a table by the values in a column, applies a reduction to each group, and puts the results back together. One line for what would otherwise be a loop, a dictionary and a concatenation.

## TWO OPERATIONS THAT WOULD OTHERWISE BE LOOPS

In engineering the grouping column is usually a run identifier, an operating point or a sensor name. Six gauges, mean and standard deviation of each: one line.

resample is the same idea on a time index, and it is how a logger's irregular ten-hertz stream becomes a regular one-second series. That is all this course says about pandas.

## ONE LINE EACH

```
# mean and spread per gauge
g = df.groupby("gauge")["strain"]
g.agg(["mean", "std", "count"])

# a regular 1 s series from an
# irregular log
reg = df.set_index("t").resample("1s").mean()

# resampling interpolates. That is a
# modelling choice, not formatting.
```

## AND THAT IS ENOUGH PANDAS

You will need about five per cent of this library. Look the rest up on the day, and do not read the documentation cover to cover.

# The State Form

**The state form** — the one form every numerical integrator accepts: a first-order system. You supply a state vector, an initial condition, and a function that takes the current time and state and returns the rate of change of each component.

## A FIRST-ORDER SYSTEM, AND A FUNCTION FOR ITS RATES

A higher-order equation is reduced to it by making the derivatives part of the state. A second-order equation becomes a two-component system of position and velocity.

So the whole of the modelling work is writing the right-hand side function. It takes  $t$  and  $y$ , in that order, and returns the derivatives, even when it does not use  $t$ .

## THE ONLY FORM A SOLVER ACCEPTS

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0$$

$$\ddot{x} = g(t, x, \dot{x}) \implies y = (x, \dot{x}), \quad \dot{y} = (\dot{x}, g)$$

## THE RIGHT-HAND SIDE, IN FULL

```
def cooling_rhs(t, y, tau, T_inf):
    """dT/dt in K/s. y = (T,)"""
    T, = y
    return [-(T - T_inf) / tau]

# t comes first, even when unused
```

# solve\_ivp

**solve\_ivp** — SciPy's initial-value solver. It takes the right-hand side, the time span as a pair, the initial state, and the argument people forget, `t_eval`, which is the times you actually want back rather than the times the stepper happened to take.

## FOUR ARGUMENTS THAT MATTER, AND TWO THAT DO NOT

The method defaults to an adaptive Runge–Kutta scheme, which is the right first choice for almost everything here. `rtol` and `atol` set the error it will accept, and the defaults are loose.

The result holds `t` and `y`, and `y` comes back as states by times. A single-state system therefore gives you shape one by `N`, which is the shape that will surprise you.

## THE CALL

```
from scipy.integrate import solve_ivp

sol = solve_ivp(
    cooling_rhs,
    t_span=(0.0, 60.0),
    y0=[80.05],
    t_eval=np.linspace(0.0, 60.0, 301),
    args=(60.0, 22.0),
    method="RK45", rtol=1e-8, atol=1e-10,
)

sol.y.shape          (1, 301)
T_num = sol.y[0]     (301,)
```

*The (1, 301) is a length-one axis. You met that trap last lecture.*

# Checking the Answer

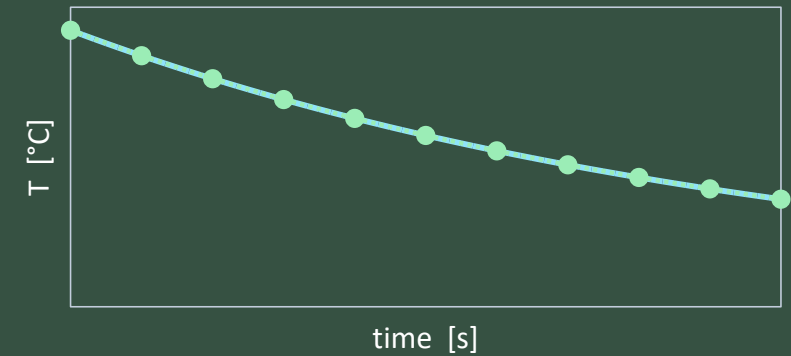
**Checking a solver** — three checks, in decreasing order of availability. Compare against an analytic solution where one exists. Check a conserved quantity, which most engineering systems have. And halve the tolerance to see whether the answer moves.

## THREE CHECKS, IN DECREASING ORDER OF AVAILABILITY

The third is the weakest and always available. If tightening the tolerance by two orders of magnitude changes your third significant figure, the answer you had was not converged.

This is not pedantry. From the eighth lecture onwards you validate physics-informed networks with these three, in this order, because there is often no test data at all.

## NUMERICAL AGAINST ANALYTIC



*Line: exponential. Dots: RK45. Max difference  $\approx 1e-8$  °C.*

## THE THREE, RANKED

|                |                                 |
|----------------|---------------------------------|
| <b>best</b>    | against an analytic case        |
| <b>usually</b> | a conserved quantity            |
| <b>always</b>  | halve the tolerance, look again |

# The Model

**Curve fitting** — choosing the parameters of a model so that it best matches measurements, which is the same three ingredients as every model in this course: a function with parameters, a measure of how wrong it is, and something that makes that measure smaller.

## THE SAME THREE INGREDIENTS AS EVERY MODEL HERE

The difference from a network is the function. Here it is a formula you wrote, with two or three parameters that mean something physical, rather than a few thousand that do not.

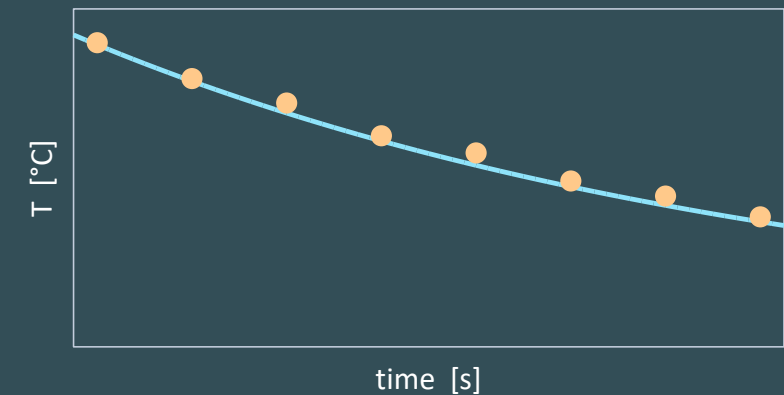
That is why a fit is the baseline a network has to beat, and why it is often the answer rather than the baseline.

$$\hat{\theta} = \operatorname{argmin}_{\theta} \sum_{k=1}^N (f(t_k, \theta) - T_k)^2$$

## THE MODEL FUNCTION

### TWO FREE PARAMETERS

```
def cooling_model(t, T0, tau):
    """T in °C. t in s. T_inf fixed."""
    return 22.0 + (T0 - 22.0) * np.exp(-t / tau)
```



*Two numbers, chosen to make the vertical distances as small as possible.*

# curve\_fit

**curve\_fit** — SciPy's least-squares fitter. Give it your model function, the measured inputs and outputs, and a starting guess, and it returns the best parameters and their covariance matrix. The starting guess matters more than people expect.

## THE ARGUMENT PEOPLE SKIP, THE OUTPUT THEY IGNORE

Pass `p0` whenever you can. Without it the optimiser starts from ones, which for a decay constant of ten to the minus four is far enough away to find nothing at all.

Pass `sigma` too when the measurements have different uncertainties, because otherwise every point is weighted the same and the noisiest one pulls hardest.

## THE CALL, AND WHAT TO DO WITH IT

```
from scipy.optimize import curve_fit

popt, pcov = curve_fit(
    cooling_model, t, T,
    p0=[80.0, 63.7],      # from physics
)

T0_hat, tau_hat = pop
perr = np.sqrt(np.diag(pcov))

tau = 59.4 +/- 1.8 s
```

$$\sigma_i = \sqrt{C_{ii}}, \quad \theta_i \pm 1.96 \sigma_i$$

*A parameter without an uncertainty is half a result.*

# The Covariance

**The covariance** — the second thing `curve_fit` returns, and the one people throw away. Its diagonal holds the variances of the fitted parameters, so the square root of each is that parameter's standard error, and the off-diagonal terms say which parameters trade off against each other.

## FOUR THINGS, AND THEY ARE NOT SMALL

A parameter with a standard error as large as itself has not been determined by your data, however good the curve looks on the plot.

Report the value and its error together. A time constant of twelve seconds means nothing until it is twelve plus or minus one, or twelve plus or minus nine.

## TWO AGAINST TWO THOUSAND

**meaning**

$\tau$  has units;  $w_{47}$  does not

**uncertainty**

sqrt of the covariance diagonal

**extrapolation**

the physics continues; a network does not

**ratio**

2 parameters, 31 samples

## AND WHAT IT DOES NOT SAY

Flexibility. If the functional form is wrong, no amount of fitting rescues it — and that is precisely the case a network is for.

# The Baseline

**The baseline** — the fitted physical model that any network you build later has to beat. It has few parameters, each with a meaning and an error bar, it extrapolates as well as its physics does, and it takes a second to fit.

## THREE MODELS, ONE DATASET, A RANKING THAT REVERSES

When it fails, it fails visibly: the residual shows structure, the covariance blows up, or the optimiser does not converge. All three are easier to notice than a network being quietly wrong.

So fit the physical model first, always, and keep its number. From the third exercise onwards that number is what every later result is compared against.

**TALK · 2 MIN** What does a fitted physical model give you that a network does not? Then, in the plot on the right, what would a network have to do before you would believe it over that curve?

## THE MEASURED NUMBERS

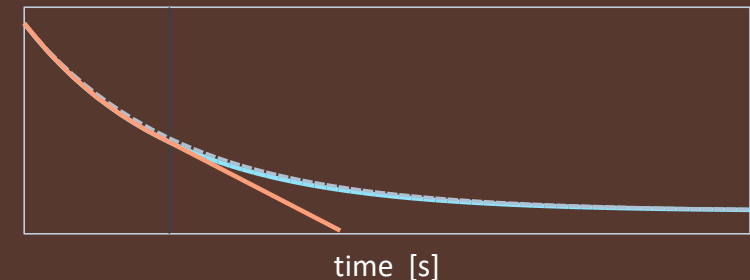
A · analytic      1.12 °C → 0.74 °C

B · curve fit      0.32 °C → 0.05 °C

C · network      0.12 °C → tens of °C

*training RMSE → extrapolation RMSE*

## WHY THE NETWORK LEAVES



# Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

| topic                | what it adds                     | read                             | where it makes a difference    |
|----------------------|----------------------------------|----------------------------------|--------------------------------|
| merge and join       | two tables keyed on a column     | pandas guide,<br>merging         | measurements against a run log |
| stiff solvers        | implicit steppers for fast modes | SciPy Integrate,<br>ODE          | chemistry, circuits, control   |
| bounded optimisation | parameters kept physical         | SciPy optimize,<br>least_squares | a rate that cannot be negative |
| interpolation        | values between the samples       | SciPy<br>interpolate             | resampling, lookup tables      |
| signal processing    | filters and spectra              | SciPy signal                     | vibration, power quality       |

WHY THEY ARE LEFT OUT

None is needed for the exercises, and each is a chapter of its own. Two will find you anyway: a stiff solver the first time a solution oscillates for no reason, and bounds the first time a fit returns a negative time constant.

# Exercise 2

## AND WHICH NOTEBOOK ACTUALLY COVERS THIS

There is no separate exercise notebook for this half-lecture, and I would rather say so than pretend otherwise. Ex02 belongs to lecture two point two and is entirely PyTorch.

The material here is exercised in Ex03, in three weeks, where you fit the cooling curve by hand and then compare it to a network. That is the right place for it, because the comparison is the point and it needs a network to compare against.

So there are two things to do now. Finish Ex01 notebooks 02 and 03 if you have not, because lecture two point two assumes both. And spend twenty minutes typing the code blocks from today's slides into a notebook of your own against any CSV you have — a lab log, a spreadsheet export, anything.

That second one is a genuine recommendation rather than a filler task. Reading a file you did not choose is the only way to meet the six problems on slide seven, and every one of them will be in there somewhere.

## WHERE THIS MATERIAL IS EXERCISED

**Ex01 02–03** finish these — L2.2 assumes them

**Ex02** next lecture, PyTorch, all of it

**Ex03 01** the curve fit, by hand

**Ex03 03** and the comparison that reverses.  
*Ex03 fits the same block again, so the number carries over.*

## REPORT QUESTION, TO CONCLUDE

Across the notebooks: give the fitted time constant with its error bar, say which of the three solver checks you ran, and state how many rows you dropped and why.

# Key Takeaways

## MEASUREMENTS

|                        |                                                     |
|------------------------|-----------------------------------------------------|
| <b>what it is</b>      | a table of labelled columns, arrays underneath      |
| <b>function</b>        | reads a real file, selects, groups, handles gaps    |
| <b>train, tune</b>     | check dtypes after every read; say what you dropped |
| <b>vs a bare array</b> | labels, a time index, and missing values            |
| <b>used for</b>        | every measurement that arrives as a file            |

## SOLVING AND FITTING

|                     |                                                   |
|---------------------|---------------------------------------------------|
| <b>what it is</b>   | an integrator and a least-squares fitter          |
| <b>function</b>     | state in, trajectory out; data in, parameters out |
| <b>train, tune</b>  | t_eval and the tolerance; p0 and sigma            |
| <b>vs a network</b> | few parameters, each with a meaning and an error  |
| <b>used for</b>     | the baseline every later model has to beat        |

# Questions

## 1 – 5 · MEASUREMENTS

- 1 What is a DataFrame, how is it stored, and what does its index give you that an array has no answer to? *pandas guide, intro*
- 2 Which six arguments of `read_csv` does a real logger file need, and what does each one fix? *pandas guide, IO; Ex\_02 notebook 01*
- 3 What is the difference between `loc` and `iloc`, and which of them includes the end of a slice? *pandas guide, indexing*
- 4 What are the three defensible answers to a missing value, and which answer is dishonest? *pandas guide, missing data*
- 5 What does `groupby` do, and what would you write instead of it if it did not exist? *pandas guide, groupby*

## 6 – 10 · SOLVING AND FITTING

- 6 What form does every integrator demand, and how do you put a second-order equation into it? *SciPy integrate*
- 7 What are the four arguments of `solve_ivp` that matter, and what shape does the result come back in? *SciPy integrate; Ex\_02 notebook 02*
- 8 What are the three checks on a numerical solution, and which one is always available? *SciPy integrate; Ex\_02 notebook 02*
- 9 What does `curve_fit` need besides the model and the data, and why does the starting guess matter? *SciPy optimize*
- 10 What is in the covariance matrix, and what does a parameter whose error equals itself tell you? *SciPy optimize; Ex\_02 notebook 03*

NEXT

## L2.2 — PyTorch as an Array Library

---

Today you differentiated nothing and solved everything by conventional means. Next time: arrays that remember what happened to them, and a differentiation engine that will still be doing the work in week twelve. It is the most load-bearing half-lecture in Part 1.

### BEFORE THE NEXT RECORDING

|               |                                    |
|---------------|------------------------------------|
| <b>finish</b> | Ex01 notebooks 02 and 03           |
| <b>try</b>    | pandas on a CSV of your own        |
| <b>read</b>   | the solve_ivp worked example       |
| <b>expect</b> | Ex02, and to spend real time on it |

*Lecture two point two is the one to watch twice.  
Everything in Part 2 depends on one function call in it,  
and the exercise notebook that goes with it is the  
most important in Part 1.*